

Practicle Exam practice

React

Note: Below all codes are Ai created

“app.js”

```
import React, { useState } from "react";
import data from "./data.json";

function App() {
  const [users, setUsers] = useState(data.users);
  const [formData, setFormData] = useState({ id: null, name: "", age: "", country: "" });
  const [isEditing, setIsEditing] = useState(false);

  // Handle input change
  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({ ...prev, [name]: value }));
  };

  // Add new user
  const addUser = () => {
    if (!formData.name || !formData.age || !formData.country) {
      alert("Please fill all fields");
      return;
    }
    const newUser = {
      id: users.length ? users[users.length - 1].id + 1 : 1,
      name: formData.name,
      age: parseInt(formData.age),
      country: formData.country,
    };
    setUsers([...users, newUser]);
    setFormData({ id: null, name: "", age: "", country: "" });
  };

  // Delete user
  const deleteUser = (id) => {
    if (window.confirm("Are you sure you want to delete this user?")) {
      setUsers(users.filter((user) => user.id !== id));
      if (isEditing && formData.id === id) {
        setIsEditing(false);
        setFormData({ id: null, name: "", age: "", country: "" });
      }
    }
  };

  // Edit user - fill form with existing data
  const editUser = (user) => {
    setIsEditing(true);
    setFormData({ id: user.id, name: user.name, age: user.age, country: user.country });
  };

  // Update user info
  const updateUser = () => {
    if (!formData.name || !formData.age || !formData.country) {
      alert("Please fill all fields");
      return;
    }
    setUsers(
```

```

        users.map((user) =>
            user.id === formData.id
                ? { ...user, name: formData.name, age: parseInt(formData.age), country: formData.country }
                : user
        )
    );
    setIsEditing(false);
    setFormData({ id: null, name: "", age: "", country: "" });
};

// Cancel editing
const cancelEdit = () => {
    setIsEditing(false);
    setFormData({ id: null, name: "", age: "", country: "" });
};

return (
    <div className="app-container">
        <h2>User Management (CRUD)</h2>

        {/* Form */}
        <div className="form-container">
            <h3>{isEditing ? "Edit User" : "Add User"}</h3>
            <input
                type="text"
                name="name"
                placeholder="Name"
                value={formData.name}
                onChange={handleChange}
            />
            <input
                type="number"
                name="age"
                placeholder="Age"
                value={formData.age}
                onChange={handleChange}
            />
            <input
                type="text"
                name="country"
                placeholder="Country"
                value={formData.country}
                onChange={handleChange}
            />
            {isEditing ? (
                <>
                    <button onClick={updateUser}>Update</button>
                    <button onClick={cancelEdit}>Cancel</button>
                </>
            ) : (
                <button onClick={addUser}>Add</button>
            )}
        </div>

        {/* Users Table */}
        <table className="users-table">
            <thead>
                <tr>
                    <th>Name</th>
                    <th>Age</th>
                    <th>Country</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody>
                {users.length === 0 ? (
                    <tr>

```

```

        <td colSpan="4">No users found.</td>
      </tr>
    ) : (
      users.map((user) => (
        <tr key={user.id}>
          <td>{user.name}</td>
          <td>{user.age}</td>
          <td>{user.country}</td>
          <td>
            <button onClick={() => editUser(user)}>Edit</button>
            <button onClick={() => deleteUser(user.id)}>Delete</button>
          </td>
        </tr>
      ))
    )
  </tbody>
</table>
</div>
);
}

export default App;

```

“data.json”

```

{
  "users": [
    {
      "id": 1,
      "name": "Alice",
      "age": 28,
      "country": "USA"
    },
    {
      "id": 2,
      "name": "Bob",
      "age": 34,
      "country": "UK"
    },
    {
      "id": 3,
      "name": "Carlos",
      "age": 25,
      "country": "Brazil"
    }
  ]
}

```

[Code Explanation]

User Management (CRUD)

Add User

Name	Age	Country	user@example.com			
.....	Select Gender ▾	Mobile Number	Add			

Name	Age	Country	Email	Password	Gender	Mobile	Actions
Tarun	25	India	tarun@example.com	tarun123	Male	9876543210	<button>Edit</button> <button>Delete</button>
Patel	28	UK	patel@example.com	patel456	Male	9123456780	<button>Edit</button> <button>Delete</button>
Riya	23	USA	riya@example.com	riya789	Female	9988776655	<button>Edit</button> <button>Delete</button>

React CRUD Application with JSON Data (Explanation)

1. Project Overview

This React application implements a **CRUD (Create, Read, Update, Delete)** functionality to manage a list of users. The application uses a **JSON file** (data.json) as the data source, meaning it works without a backend server. Users can **add, edit, update, and delete** data directly in the app.

2. Data Structure (data.json)

The JSON file stores the user data in a structured way. Here is the example structure:

```
{
  "users": [
    {
      "id": 1,
      "name": "Tarun",
      "age": 25,
      "country": "India",
      "email": "tarun@example.com",
      "password": "tarun123",
      "gender": "Male",
      "mobile": "9876543210"
    },
    {
      "id": 2,
      "name": "Patel",
      "age": 28,
      "country": "UK",

```

```

    "email": "patel@example.com",
    "password": "patel456",
    "gender": "Male",
    "mobile": "9123456780"
  },
  {
    "id": 3,
    "name": "Riya",
    "age": 23,
    "country": "USA",
    "email": "riya@example.com",
    "password": "riya789",
    "gender": "Female",
    "mobile": "9988776655"
  }
]
}

```

Explanation of fields:

- id → Unique identifier for each user.
 - name → User's name.
 - age → User's age.
 - country → User's country.
 - email → User's email address.
 - password → User's password.
 - gender → User's gender.
 - mobile → User's mobile number.
-

3. React Component (App.js)

Imports

```

import React, { useState } from "react";
import data from "./data.json";

```

- useState → React Hook for managing state.
 - data → Imports the initial JSON data from data.json.
-

State Variables

```

const [users, setUsers] = useState(data.users);
const [formData, setFormData] = useState({ id: null, name: "", age: "", country: "", email: "", password: "", gender: "", mobile: "" });
const [isEditing, setIsEditing] = useState(false);

```

- users → Stores all users from JSON or newly added users.
 - formData → Stores current form input values for add/edit.
 - isEditing → Tracks whether the form is in editing mode.
-

Handle Input Change

```
const handleChange = (e) => {  
  const { name, value } = e.target;  
  setFormData((prev) => ({ ...prev, [name]: value }));  
};
```

- Updates formData whenever a user types in an input field.
 - Uses name attribute to dynamically update the corresponding field.
-

Add New User

```
const addUser = () => {  
  if (!formData.name || !formData.age || !formData.country || !formData.email || !formData.password ||  
  !formData.gender || !formData.mobile) {  
    alert("Please fill all fields");  
    return;  
  }  
  const newUser = {  
    id: users.length ? users[users.length - 1].id + 1 : 1,  
    ...formData,  
    age: parseInt(formData.age),  
  };  
  setUsers([...users, newUser]);  
  setFormData({ id: null, name: "", age: "", country: "", email: "", password: "", gender: "", mobile: "" });  
};
```

- Validates that all fields are filled.
 - Creates a new user object.
 - Adds the new user to the users array.
 - Resets the form fields after adding.
-

Edit User

```
const editUser = (user) => {  
  setIsEditing(true);  
  setFormData({ ...user });  
};
```

- Fills the form with the selected user's data for editing.
 - Sets isEditing to true.
-

Update User

```
const updateUser = () => {  
  if (!formData.name || !formData.age || !formData.country || !formData.email || !formData.password ||  
  !formData.gender || !formData.mobile) {  
    alert("Please fill all fields");  
    return;  
  }  
  setUsers(  
    users.map((user) =>  
      user.id === formData.id ? { ...formData, age: parseInt(formData.age) } : user
```

```

    )
  );
  setIsEditing(false);
  setFormData({ id: null, name: "", age: "", country: "", email: "", password: "", gender: "", mobile: "" });
};

```

- Validates all fields.
- Updates the selected user's information in users.
- Resets the form and exits edit mode.

Delete User

```

const deleteUser = (id) => {
  if (window.confirm("Are you sure you want to delete this user?")) {
    setUsers(users.filter((user) => user.id !== id));
    if (isEditing && formData.id === id) {
      setIsEditing(false);
      setFormData({ id: null, name: "", age: "", country: "", email: "", password: "", gender: "", mobile: "" });
    }
  }
};

```

- Confirms deletion with the user.
- Removes the selected user from the users array.
- Cancels editing if the deleted user is being edited.

Cancel Edit

```

const cancelEdit = () => {
  setIsEditing(false);
  setFormData({ id: null, name: "", age: "", country: "", email: "", password: "", gender: "", mobile: "" });
};

```

- Resets the form fields and exits editing mode.

Render Form and Table

```

<div className="app-container">
  <h2>User Management (CRUD)</h2>

  { /* Form */ }
  <div className="form-container">
    <h3>{isEditing ? "Edit User" : "Add User"}</h3>
    <input type="text" name="name" placeholder="Name" value={formData.name}
    onChange={handleChange} />
    <input type="number" name="age" placeholder="Age" value={formData.age}
    onChange={handleChange} />
    <input type="text" name="country" placeholder="Country" value={formData.country}
    onChange={handleChange} />
    <input type="email" name="email" placeholder="Email" value={formData.email}
    onChange={handleChange} />

```

```

    <input type="password" name="password" placeholder="Password" value={formData.password}
onChange={handleChange} />
    <input type="text" name="gender" placeholder="Gender" value={formData.gender}
onChange={handleChange} />
    <input type="text" name="mobile" placeholder="Mobile" value={formData.mobile}
onChange={handleChange} />
    {isEditing ? (
      <>
        <button onClick={updateUser}>Update</button>
        <button onClick={cancelEdit}>Cancel</button>
      </>
    ) : (
      <button onClick={addUser}>Add</button>
    )}
  </div>

```

```

{/* Users Table */}
<table className="users-table">
  <thead>
    <tr>
      <th>Name</th>
      <th>Age</th>
      <th>Country</th>
      <th>Email</th>
      <th>Password</th>
      <th>Gender</th>
      <th>Mobile</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody>
    {users.length === 0 ? (
      <tr>
        <td colspan="8">No users found.</td>
      </tr>
    ) : (
      users.map((user) => (
        <tr key={user.id}>
          <td>{user.name}</td>
          <td>{user.age}</td>
          <td>{user.country}</td>
          <td>{user.email}</td>
          <td>{user.password}</td>
          <td>{user.gender}</td>
          <td>{user.mobile}</td>
          <td>
            <button onClick={() => editUser(user)}>Edit</button>
            <button onClick={() => deleteUser(user.id)}>Delete</button>
          </td>
        </tr>
      ))
    )}
  </tbody>
</table>
</div>

```

Explanation:

- Shows a **form** to add or edit users.

- Dynamically changes the heading and buttons based on `isEditing`.
 - Displays all users in a **table**, including the new fields: email, password, gender, mobile.
 - Each row has **Edit** and **Delete** buttons to manage the user.
-

4. Key Features

1. Works entirely on **frontend** using React state.
2. Data is initially loaded from **data.json**.
3. Supports full **CRUD operations**:
 - **Add**: New users can be added with all fields.
 - **Edit/Update**: Existing users can be updated.
 - **Delete**: Users can be removed.
4. Includes **form validation** to ensure no empty fields.
5. Dynamic rendering using `map()` and conditional JSX (`isEditing`).

103:-Angular

Note:-

1.This code not Error But not display db.json data on this data so carefully use.

2.I don't know how code work on other Pc or Laptop/Device

“This I created”

ng new practice1

✓ Do you want to create a 'zoneless' application without zone.js (Developer Preview)? No

✓ Which stylesheet format would you like to use? CSS [
<https://developer.mozilla.org/docs/Web/CSS>]

✓ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No

CREATE practice1/angular.json (2497 bytes)

CREATE practice1/package.json (1131 bytes)

CREATE practice1/README.md (1531 bytes)

CREATE practice1/tsconfig.json (1026 bytes)

CREATE practice1/.editorconfig (331 bytes)

CREATE practice1/.gitignore (629 bytes)

CREATE practice1/tsconfig.app.json (444 bytes)

CREATE practice1/tsconfig.spec.json (422 bytes)

CREATE practice1/.vscode/extensions.json (134 bytes)

CREATE practice1/.vscode/launch.json (490 bytes)

CREATE practice1/.vscode/tasks.json (980 bytes)

CREATE practice1/src/main.ts (228 bytes)

CREATE practice1/src/index.html (308 bytes)

CREATE practice1/src/styles.css (81 bytes)

CREATE practice1/src/app/app.spec.ts (690 bytes)

CREATE practice1/src/app/app.ts (303 bytes)

CREATE practice1/src/app/app.css (0 bytes)

CREATE practice1/src/app/app.html (20438 bytes)

CREATE practice1/src/app/app.config.ts (412 bytes)

CREATE practice1/src/app/app.routes.ts (80 bytes)

CREATE practice1/public/favicon.ico (15086 bytes)

- Installing packages (npm)...(node:23532) [DEP0190]

DeprecationWarning: Passing args to a child process with shell option

true can lead to security vulnerabilities, as the arguments are not escaped, only concatenated.

(Use ``node --trace-deprecation ...`` to show where the warning was created)

✓ Packages installed successfully.

Successfully initialized git.

User Management (CRUD)

Add User

Name	Age	Country	Mobile				
Gender	Email	Password	Add				
Name	Age	Country	Mobile	Gender	Email	Password	Actions
Tarunbhai Champakbhai Patel	10	India	09875239438	male	18patel70@gmail.com	1234	<button>Edit</button> <button>Delete</button>

Application tab showing Local Storage. The 'users' key contains an array of three user objects: {id: 1, name: 'Tarun', age: 25, country: 'India'}, {id: 2, name: 'Ravi', age: 30, country: 'USA'}, and {id: 3, name: 'tarun', age: 20, country: 'valsad'}.

“angular.json” add-

```
    "assets": [  
      {  
        "glob": "**/*",  
        "input": "src/assets",  
        "output": "/assets"  
      },  
      {  
        "glob": "**/*",  
        "input": "public",  
        "output": "/"  
      }  
    ],  
  },  
  "main.ts"
```

“main.ts”

```
// main.ts  
import { bootstrapApplication } from '@angular/platform-browser';  
import { importProvidersFrom } from '@angular/core';  
import { HttpClientModule } from '@angular/common/http';  
import { AppComponent } from './app/app';
```

```
bootstrapApplication(AppComponent, {  
  providers: [importProvidersFrom(HttpClientModule)]  
}).catch(err => console.error(err));
```

“db.json”

```

{
  "users": [
    {
      "id": 1,
      "name": "Tarun",
      "age": 25,
      "country": "India",
      "mobile": "9876543210",
      "gender": "Male",
      "email": "tarun@example.com",
      "password": "tarun123"
    },
    {
      "id": 2,
      "name": "Patel",
      "age": 28,
      "country": "UK",
      "mobile": "9123456780",
      "gender": "Male",
      "email": "patel@example.com",
      "password": "patel123"
    }
  ]
}

```

“app.ts”

```

import { Component, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { CommonModule } from '@angular/common';
import { HttpClient } from '@angular/common/http';

interface User {
  id: number;
  name: string;
  age: number;
  country: string;
  mobile: string;
  gender: string;
  email: string;
  password: string;
}

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class AppComponent implements OnInit {
  users: User[] = [];
  formData: Partial<User> = {};
  isEditing = false;

  constructor(private http: HttpClient) {}

  ngOnInit() {
    // Load initial data from db.json in assets
    this.http.get<{ users: User[] }>('assets/db.json').subscribe({
      next: (data) => this.users = data.users,
      error: (err) => console.error('Error loading db.json:', err)
    });
  }
}

```

```

addUser() {
  if (!this.formData.name || !this.formData.age || !this.formData.country
    || !this.formData.mobile || !this.formData.gender
    || !this.formData.email || !this.formData.password) {
    alert('Please fill all fields');
    return;
  }

  const newUser: User = {
    id: this.users.length ? this.users[this.users.length - 1].id + 1 : 1,
    name: this.formData.name,
    age: Number(this.formData.age),
    country: this.formData.country,
    mobile: this.formData.mobile!,
    gender: this.formData.gender!,
    email: this.formData.email!,
    password: this.formData.password!
  };

  this.users.push(newUser);
  this.formData = {};
}

editUser(user: User) {
  this.isEditing = true;
  this.formData = { ...user };
}

updateUser() {
  if (!this.formData.name || !this.formData.age || !this.formData.country
    || !this.formData.mobile || !this.formData.gender
    || !this.formData.email || !this.formData.password) {
    alert('Please fill all fields');
    return;
  }

  this.users = this.users.map(u =>
    u.id === this.formData.id ? { ...u, ...this.formData } as User : u
  );
  this.isEditing = false;
  this.formData = {};
}

cancelEdit() {
  this.isEditing = false;
  this.formData = {};
}

deleteUser(id: number) {
  if (confirm('Are you sure you want to delete this user?')) {
    this.users = this.users.filter(u => u.id !== id);
    if (this.isEditing && this.formData.id === id) this.cancelEdit();
  }
}
}

```

“app.html”

```

<div class="app-container">
  <h2>User Management (CRUD)</h2>

  <!-- Form -->
  <div class="form-container">
    <h3>{{ isEditing ? 'Edit User' : 'Add User' }}</h3>
    <input type="text" placeholder="Name" [(ngModel)]="formData.name" />

```

```

<input type="number" placeholder="Age" [(ngModel)]="formData.age" />
<input type="text" placeholder="Country" [(ngModel)]="formData.country" />
<input type="text" placeholder="Mobile" [(ngModel)]="formData.mobile" />
<input type="text" placeholder="Gender" [(ngModel)]="formData.gender" />
<input type="email" placeholder="Email" [(ngModel)]="formData.email" />
<input type="password" placeholder="Password" [(ngModel)]="formData.password" />

<button *ngIf="!isEditing" (click)="addUser()">Add</button>
<button *ngIf="isEditing" (click)="updateUser()">Update</button>
<button *ngIf="isEditing" (click)="cancelEdit()">Cancel</button>
</div>

<!-- Users Table -->
<table class="users-table" border="1" cellpadding="5" cellspacing="0">
  <thead>
    <tr>
      <th>Name</th>
      <th>Age</th>
      <th>Country</th>
      <th>Mobile</th>
      <th>Gender</th>
      <th>Email</th>
      <th>Password</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody>
    <tr *ngIf="users.length === 0">
      <td colspan="8">No users found.</td>
    </tr>
    <tr *ngFor="let user of users">
      <td>{{ user.name }}</td>
      <td>{{ user.age }}</td>
      <td>{{ user.country }}</td>
      <td>{{ user.mobile }}</td>
      <td>{{ user.gender }}</td>
      <td>{{ user.email }}</td>
      <td>{{ user.password }}</td>
      <td>
        <button (click)="editUser(user)">Edit</button>
        <button (click)="deleteUser(user.id)">Delete</button>
      </td>
    </tr>
  </tbody>
</table>
</div>

```

[Code Explanation]

Angular CRUD Application Using db.json (Offline / Frontend Only)

1. db.json File

```

{
  "users": [
    {
      "id": 1,
      "name": "Tarun",
      "age": 25,

```

```

    "country": "India",
    "mobile": "9876543210",
    "gender": "Male",
    "email": "tarun@example.com",
    "password": "tarun123"
  },
  {
    "id": 2,
    "name": "Patel",
    "age": 28,
    "country": "UK",
    "mobile": "9123456780",
    "gender": "Male",
    "email": "patel@example.com",
    "password": "patel123"
  }
]
}

```

Explanation:

- db.json acts as a static “fake backend” file.
- The users array stores user objects.
- Each user has the following fields:
 - id → unique identifier.
 - name, age, country → basic details.
 - mobile, gender, email, password → additional fields.
- Located in src/assets/db.json to make it accessible via Angular HTTP requests.

2. AppComponent (app.ts)

```

import { Component, OnInit } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { CommonModule } from '@angular/common';
import { HttpClient } from '@angular/common/http';

```

```

interface User {
  id: number;
  name: string;
  age: number;
  country: string;
  mobile?: string;
  gender?: string;
  email?: string;
  password?: string;
}

```

```

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class AppComponent implements OnInit {

```

```

users: User[] = [];
formData: Partial<User> = {};
isEditing = false;

constructor(private http: HttpClient) {}

ngOnInit() {
  // Load initial data from db.json in assets folder
  this.http.get<{ users: User[] }>('assets/db.json').subscribe({
    next: (data) => this.users = data.users,
    error: (err) => console.error('Error loading db.json:', err)
  });
}

addUser() {
  if (!this.formData.name || !this.formData.age || !this.formData.country) {
    alert('Please fill all required fields.');
```

Explanation:

- **Imports:**
 - FormsModule → for two-way binding using [(ngModel)].
 - CommonModule → for Angular directives like *ngFor and *ngIf.
 - HttpClient → to load db.json from assets.
- **Interface User:** Defines user structure including new fields (mobile, gender, email, password).
- **Variables:**
 - users → stores all users from db.json.
 - formData → temporary object for the Add/Edit form.
 - isEditing → tracks whether the form is in Add or Edit mode.
- **ngOnInit:** Loads initial users from db.json. Logs an error if the file is missing.
- **addUser:** Adds a new user to the array with auto-incremented id.
- **editUser:** Populates the form with selected user's data for editing.
- **updateUser:** Updates the existing user in the users array.
- **cancelEdit:** Cancels editing and clears the form.
- **deleteUser:** Removes a user by id. If the deleted user was being edited, editing is canceled.

3. app.html (Template)

```
<div class="app-container">
  <h2>User Management (CRUD)</h2>

  <!-- Form -->
  <div class="form-container">
    <h3>{{ isEditing ? 'Edit User' : 'Add User' }}</h3>
    <input type="text" placeholder="Name" [(ngModel)]="formData.name" />
    <input type="number" placeholder="Age" [(ngModel)]="formData.age" />
    <input type="text" placeholder="Country" [(ngModel)]="formData.country" />
    <input type="text" placeholder="Mobile" [(ngModel)]="formData.mobile" />
    <select [(ngModel)]="formData.gender">
      <option value="">Select Gender</option>
      <option value="Male">Male</option>
      <option value="Female">Female</option>
    </select>
    <input type="email" placeholder="Email" [(ngModel)]="formData.email" />
    <input type="password" placeholder="Password" [(ngModel)]="formData.password" />

    <button *ngIf="!isEditing" (click)="addUser()">Add</button>
    <button *ngIf="isEditing" (click)="updateUser()">Update</button>
    <button *ngIf="isEditing" (click)="cancelEdit()">Cancel</button>
  </div>

  <!-- Users Table -->
  <table class="users-table" border="1" cellpadding="5" cellspacing="0">
    <thead>
      <tr>
        <th>Name</th>
        <th>Age</th>
        <th>Country</th>
        <th>Mobile</th>
        <th>Gender</th>
        <th>Email</th>
      </tr>
    </thead>
  </table>
</div>
```

```

        <th>Password</th>
        <th>Actions</th>
    </tr>
</thead>
<tbody>
    <tr *ngIf="users.length === 0">
        <td colspan="8">No users found.</td>
    </tr>
    <tr *ngFor="let user of users">
        <td>{{ user.name }}</td>
        <td>{{ user.age }}</td>
        <td>{{ user.country }}</td>
        <td>{{ user.mobile }}</td>
        <td>{{ user.gender }}</td>
        <td>{{ user.email }}</td>
        <td>{{ user.password }}</td>
        <td>
            <button (click)="editUser(user)">Edit</button>
            <button (click)="deleteUser(user.id)">Delete</button>
        </td>
    </tr>
</tbody>
</table>
</div>

```

Explanation:

- The form allows **adding or editing** a user with all fields.
- *ngIf controls the **Add / Update / Cancel** buttons.
- The table displays all user fields dynamically.
- Edit/Delete buttons call respective methods in the component.

4. Key Points

1. db.json is a **static JSON file**, so changes made in the app are **not saved to the file**.
2. CRUD operations are **client-side only**. For persistence, use **localStorage** or a backend.
3. HttpClientModule is required in **main.ts or AppModule** if you are not using standalone components.
4. Angular directives like *ngFor and *ngIf make dynamic rendering easy.
5. Two-way binding ([[ngModel]]) allows form inputs to bind directly to the component variables.

This is **ready to copy-paste into Word**, with proper formatting for a project report or documentation.

If you want, I can **also add a diagram showing flow**: Add → Edit → Update → Delete → Table display, which makes it **perfect for a report**.

